Toy network demo

Table of contents

1. The network dataset 1
2. Build the tbnpy model 6
3. The RSR results 8
4. Sample the BN 8

This is an illustration using the toy network `toy-ca`. The only user input required is `network.json`, which is explained in Section 1. The rest of the demo is automated.

1. The network dataset

The network is one file, `examples/toy-ca/network.json`, with one table per entity type.

```
data = json.loads(Path("examples/toy-ca/network.json").read_text(encoding="utf-8"))
{table: len(rows) for table, rows in data["tables"].items()}
```

```
{'Node': 8, 'Edge': 10, 'Bridge': 10, 'Event': 1}
```

One row of each table:

```
for table in ("Node", "Edge", "Bridge", "Event"):
    pprint({table: data["tables"][table][0]}, width=85, sort_dicts=False)
```

```
{'Node': {'id': 'n1',
          'lat': 36.2754377087278,
          'lon': -120.2697236285474,
          'key_destination': True}}
{'Edge': {'id': 'e1',
          'source': 'n1',
          'target': 'n2',
          'directed': False,
          'length_km': 106.011906,
          'capacity': 1,
          'avg_speed_kph': 60}}
{'Bridge': {'id': 'b1',
            'lat': 36.66025089106477,
            'lon': -120.62045329929501,
            'edge': 'e1',
            'hazus_class': 'HWB11'}}
{'Event': {'id': 'demo-san-andreas',
           'mw': 7.8,
           'lat': 36.5,
           'lon': -121.04,
           'depth_km': 10.0,
           'description': 'Hypothetical M7.8 on the San Andreas trace, at the '
                          "network's centre latitude"}}
```

The tables:

```
import pandas as pd

nodes = pd.DataFrame(data["tables"]["Node"]).set_index("id")
edges = pd.DataFrame(data["tables"]["Edge"]).set_index("id")
bridges = pd.DataFrame(data["tables"]["Bridge"]).set_index("id")
nodes
```

id	lat	lon	key_destination
n1	36.275438	-120.269724	True
n2	37.045064	-120.971183	True
n3	36.830997	-121.387529	False
n4	36.503901	-121.127879	False
n5	36.177421	-120.827670	False
n6	36.112167	-121.022557	False
n7	36.409971	-121.328475	False
n8	36.666199	-121.645813	False

```
edges
```

id	source	target	directed	length_km	capacity	avg_speed_kph
e1	n1	n2	False	106.011906	1	60
e2	n1	n5	False	51.220587	1	60
e3	n2	n3	False	43.998194	1	60
e4	n3	n4	False	43.118452	1	60
e5	n3	n8	False	29.417095	1	60
e6	n4	n5	False	45.176621	1	60
e7	n4	n7	False	20.759042	1	60
e8	n5	n6	False	18.944186	1	60
e9	n6	n7	False	42.998648	1	60
e10	n7	n8	False	40.193808	1	60

```
bridges
```

id	lat	lon	edge	hazus_class
b1	36.660251	-120.620453	e1	HWB11
b2	36.226429	-120.548697	e2	HWB5
b3	36.938031	-121.179356	e3	HWB11
b4	36.667449	-121.257704	e4	HWB5
b5	36.748598	-121.516671	e5	HWB11
b6	36.340661	-120.977775	e6	HWB5
b7	36.456936	-121.228177	e7	HWB11
b8	36.144794	-120.925114	e8	HWB11
b9	36.261069	-121.175516	e9	HWB11
b10	36.538085	-121.487144	e10	HWB11

The schemas the file is checked against: its shape, from its `$schema`, and each table's columns, from `schema/entities.json`.

```
from jsonschema import Draft202012Validator

shape = json.loads(Path("schema/meta/network.schema.json").read_text(encoding="utf-8"))
print("$schema:", data["$schema"])
print("shape errors:", list(Draft202012Validator(shape).iter_errors(data)))
```

```
$schema: ../../schema/meta/network.schema.json
shape errors: []
```

```
entities = {e["id"]: e for e in
 ↪  json.loads(Path("schema/entities.json").read_text(encoding="utf-8"))["entities"]}

columns = []
for table, records in data["tables"].items():
    attrs = entities[table]["attributes"]
    in_file = set().union(*(r.keys() for r in records)) - {"id"}
    for name, spec in attrs.items():
        columns.append({"table": table, "column": name,
                        "schema type": f"-> {spec['ref']}" if "ref" in spec else
                         ↪  spec["type"],
                        "required": spec.get("required", True), "in file": name in in_file})
    for name in sorted(in_file - attrs.keys()):
        columns.append({"table": table, "column": name, "schema type": "NOT IN SCHEMA",
 ↪  "required": None, "in file": True})
columns = pd.DataFrame(columns)
columns
```

	table	column	schema type	required	in file
0	Node	lat	number	True	True
1	Node	lon	number	True	True
2	Node	population	number	False	False
3	Node	name	string	False	False
4	Node	key_destination	boolean	True	True
5	Edge	source	-> Node	True	True
6	Edge	target	-> Node	True	True
7	Edge	directed	boolean	True	True
8	Edge	length_km	number	True	True
9	Edge	capacity	integer	True	True
10	Edge	avg_speed_kph	number	True	True
11	Bridge	lat	number	True	True
12	Bridge	lon	number	True	True
13	Bridge	edge	-> Edge	True	True
14	Bridge	hazus_class	string	True	True
15	Bridge	vs30	number	False	False
16	Bridge	vs30_source	string	False	False
17	Event	mw	number	False	True
18	Event	lat	number	False	True
19	Event	lon	number	False	True
20	Event	depth_km	number	False	True
21	Event	description	string	False	True

They match: every required column is in the file, and the file has no column the schema lacks.

```
missing = columns[columns.required.eq(True) & ~columns["in file"]]
extra = columns[columns["schema type"] == "NOT IN SCHEMA"]
print("required but missing:", missing[["table", "column"]].values.tolist())
print("in file, not in schema:", extra[["table", "column"]].values.tolist())
```

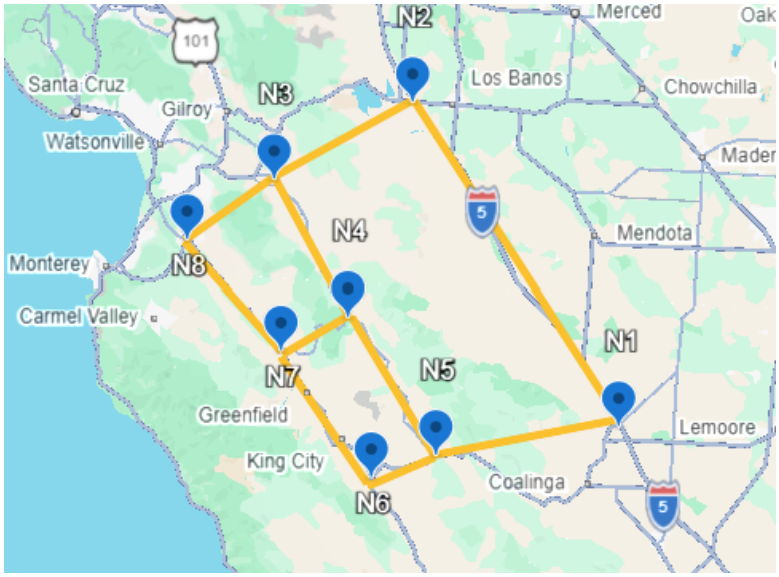
```
required but missing: []
in file, not in schema: []
```

Every assumed value is named in the file:

```
for key, text in data["assumptions"].items():
    print(fill(f"- {key}: {text}", width=90, subsequent_indent="  "))
```

```
- direction: All connections are treated as undirected.
- length_km: Great-circle distance between endpoint coordinates using a spherical Earth
  radius of 6371.0088 km; not road distance.
- capacity: Each edge has placeholder capacity 1 in arbitrary units; no measured
  capacities were supplied.
- bridge_representation: One bridge per edge; these are synthetic bridges for the toy
  example.
- bridge_location: Each bridge is placed at the arithmetic mean of its edge endpoints
  latitude and longitude; not a surveyed location.
- bridge_class: Seven bridges HWB11 (seismically designed continuous concrete); b2, b4 and
  b6, the three nearest the San Andreas trace, HWB5 (older, non-seismic) for the demo. Toy
  assumptions, not an inventory classification.
- avg_speed_kph: 60 km/h on every edge, assumed (the value the rsr and ndtools demos use);
  no measured speeds are in the dataset.
- key_destination: Key destinations n1 and n2, as set by the user.
- event: Hypothetical M7.8 as a point source on the San Andreas trace at 36.50 N, 121.04
  W, 10 km deep, for the demo only. This is the creeping section, not the 1857 rupture; a
  point source understates shaking along a ~300 km rupture.
```

The map:




```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(6, 5))
for _, e in edges.iterrows():
    a, b = nodes.loc[e.source], nodes.loc[e.target]
    ax.plot([a.lon, b.lon], [a.lat, b.lat], color="0.7", zorder=1)
ax.scatter(nodes.lon, nodes.lat, s=40, color="k", zorder=2, label="node")
for cls, group in bridges.groupby("hazus_class"):
    ax.scatter(group.lon, group.lat, s=35, marker="s", zorder=3, label=f"bridge {cls}")
key = nodes[nodes.key_destination]
ax.scatter(key.lon, key.lat, s=160, facecolors="none", edgecolors="tab:green", linewidths=2,
 ↪  zorder=4,
```

```
            label="key destination")
event = data["tables"]["Event"][0]
ax.scatter(event["lon"], event["lat"], marker="*", s=300, color="orange", zorder=5,
 ↪  label=f"scenario M{event['mw']}")
for nid, n in nodes.iterrows():
    ax.annotate(nid, (n.lon, n.lat), xytext=(4, 4), textcoords="offset points")
ax.set_aspect(1 / np.cos(np.radians(nodes.lat.mean())))
ax.set_xlabel("longitude")
ax.set_ylabel("latitude")
ax.legend(loc="lower left", fontsize=8)
plt.show()
```

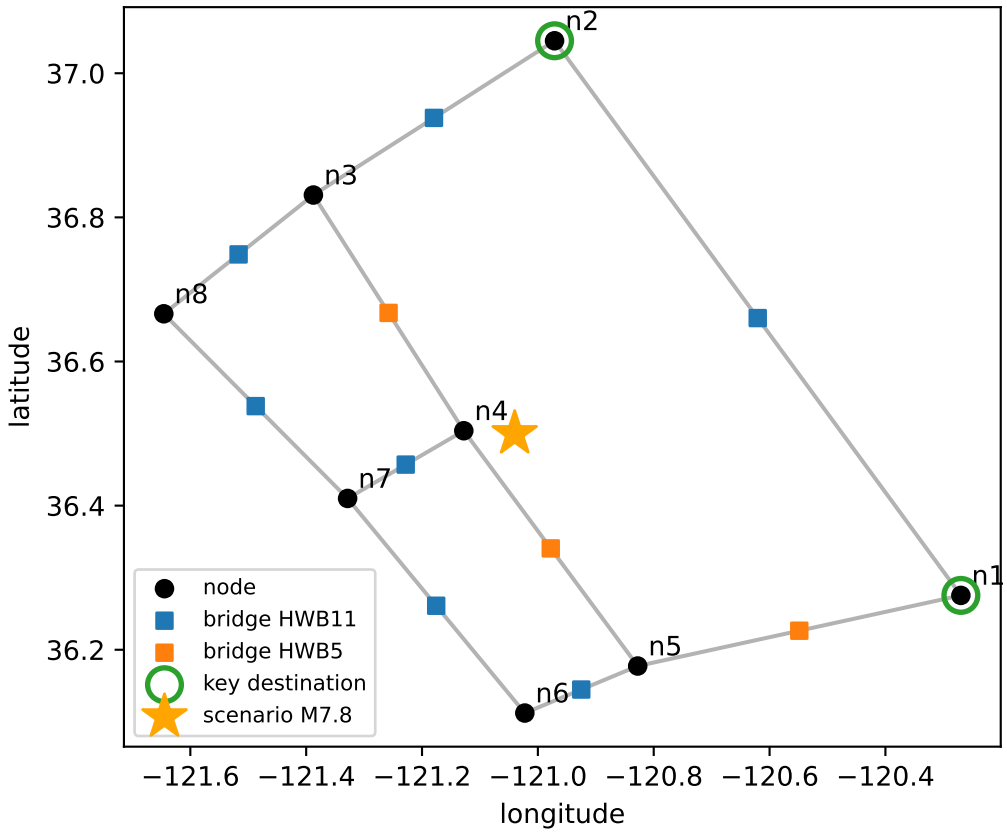


Figure 1: Nodes, edges, bridges by class, key destinations and the demo scenario.

Loading the network checks every row against `schema/entities.json`:

```
from src.builder.load import check_tables, load_library, load_network

lib = load_library()                        # schema/ and instances/
net = load_network("toy-ca", lib)
{entity: len(rows) for entity, rows in net.tables.items()}
```

```
{'Node': 8, 'Edge': 10, 'Bridge': 10, 'Event': 1}
```

A broken row is refused, with the reason:

```
broken = json.loads(json.dumps(net.tables))
broken["Bridge"][0]["edge"] = "e99"
broken["Edge"][0].pop("avg_speed_kph")
_, problems = check_tables(broken, lib, "toy-ca (edited)")
problems
```

```
["toy-ca (edited): Edge 'e1': missing required attribute 'avg_speed_kph'",
 "toy-ca (edited): Bridge 'b1': edge = 'e99' is not a Edge row"]
```

2. Build the tbnpy model

The builder resolves the assembly and saves the model and RSR rules under this example.

```
from src.builder.bn_builder import (
    generate_tbn, define_tbn_variables, define_tbn_probabilities,
)

model_dir = generate_tbn(Path("examples/toy-ca/demo.assembly.json"))
```

Load the saved data into tbnpy's two dictionaries:

```
model_data = json.loads((model_dir / "model.json").read_text(encoding="utf-8"))
varis = define_tbn_variables(model_data)
probs = define_tbn_probabilities(model_data, varis, directory=model_dir)
print(f"{len(varis)} variables; {len(probs)} probability objects")
```

```
52 variables; 6 probability objects
```

`varis` holds scalar variables, grouped below. The scenario (the Event row) is fixed data read by the hazard object, so it is not a variable.

quantity	count	type
SA(1.0)	10	continuous
DamageState.HazusBridge	10	discrete
Usable	10	discrete
EdgeOpen	10	discrete
TravelTime	6	continuous
SystemState	6	discrete

Total: **16 continuous** and **36 discrete** variables.

`probs` holds the numerical functions, each with `childs`, `parents`, `sample()` and `log_prob()`.

```
show_table(pd.DataFrame([
    {"key": k, "class": type(p).__name__,
     "children": len(p.childs), "parents": len(p.parents)}
    for k, p in probs.items()
]))
```

key	class	children	parents
hazard.gmm	Gmm	10	0
fragility.bridge	Fragility	10	10
functionality.usable-from-damage	UsableFromDamage	10	10
edge.open	EdgeOpen	10	10
system.travel-time-to-nearest:travel_time	TravelTimeToNearest	6	10
system.travel-time-to-nearest:state	TravelTimeToNearest	6	10

The graph groups variables by quantity. Arrows show the conditional direction; travel time and system state both take edge states as inputs.

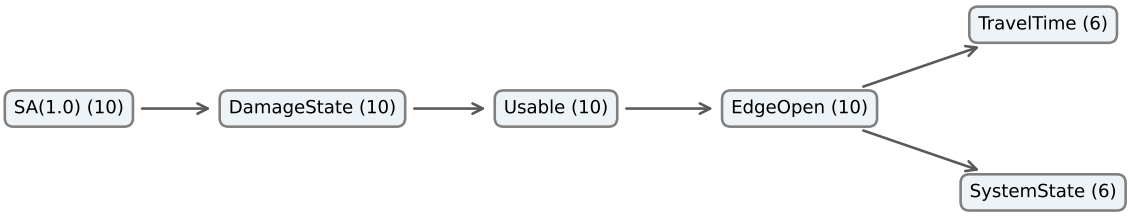


Figure 2: Variable families and their scalar counts. Each family is calculated by one probability object.

One instance of each type

In tbnpy, a probability object's `parents` are its inputs and `childs` are its outputs. Objects batch variables together; the examples below follow bridge b4, edge e4 and origin n4.

Intensity — **`SA(1.0)::b4.`** No parents: the event's Mw, epicentre and depth are fixed data from `network.json`. Outputs: all ten bridge intensities jointly; b4's output feeds `DamageState.HazusBridge::b4`. The Campbell function calculates log-median μ and dispersion σ, then draws $I = \exp(\mu + \sigma Z)$ with $Z \sim N(0, 1)$. This demo supplies only SA(1.0), with independent residuals between bridges.

For b4: median = 0.282 g; log-dispersion = 0.620; sampled intensity = 0.200 g.

Damage — **`DamageState.HazusBridge::b4.`** The object takes ten intensities and produces ten damage states; b4 uses its own intensity. Its output feeds `Usable::b4`. Lognormal exceedances are differenced into five state probabilities, then `torch.multinomial` draws a state index. Indices 0–4 denote none through complete.

damage state	P(state given sampled intensity)
none	0.645
slight	0.179
moderate	0.087
extensive	0.070
complete	0.018

Sampled damage at b4: **none**.

Usability — **`Usable::b4.`** Parents: ten damage variables. Outputs: ten usability variables; b4's output feeds `EdgeOpen::e4`. The closure policy deterministically maps none/slight/moderate to 1, and extensive/complete to 0.

Edge state — **`EdgeOpen::e4.`** Parents: the bridge usability variables. Outputs: ten edge states. An edge is open only if every bridge on it is usable; e4 has bridge b4. Edge states feed both system objects, for origins n3–n8.

For example, this calls the edge object's sampling method directly. The tensor columns follow `parents`; returned columns follow `childs`.

```
edge = prob_of("edge.open")
Cs_pars = example_parents(edge)
Cs_child, logp = edge.sample(Cs_pars=Cs_pars)
j = next(i for i, v in enumerate(edge.childs) if v.name == "EdgeOpen::e4")
print("b4 usability:", int(sampled_value("Usable::b4")))
print("e4 state:", int(Cs_child[0, j]), "; log probability:", float(logp[0]))
```

```
b4 usability: 1
e4 state: 1 ; log probability: 0.0
```

Travel time — `TravelTime::n4`. Parents: all ten edge states. Outputs: six origin travel times, with no downstream variables. For each distinct edge configuration, ndtools finds the shortest open route to n1 or n2, using edge time = length/speed. Disconnected origins receive infinity.

System state — `SystemState::n4`. The same ten edge parents produce six binary states, with no downstream variables. RSR first classifies the edge configuration; ndtools handles any unknowns. Survival means $T_{n4} \leq T_{n4}^{\text{intact}} + 0.5$ hours. The rule classifier determines this event directly from edges.

variable	sampled value
TravelTime::n4	1.452
SystemState::n4	1.000

3. The RSR results

Each origin has survival and failure rules under `rsr/<origin>/`. The counts below also check rule coverage across all 1,024 edge configurations.

origin	survival rules	failure rules	rounds	unclassified / 1024
n3	1	1	3	0
n4	2	4	7	0
n5	1	1	3	0
n6	1	2	4	0
n7	4	16	20	2
n8	1	2	4	0

For example, these rules determine whether n4 stays within 0.5 hours of its intact travel time. `1` means open; `0` means closed. Unlisted edges are unrestricted.

```
for filename, outcome in (("refs_up_1.json", "survives"),
                          ("refs_low_0.json", "fails")):
    rules = json.loads((model_dir / "rsr/n4" / filename).read_text())
    for rule in rules:
        conditions = " and ".join(f"{edge}={value}" for edge, (_, value) in rule.items()
                                  if edge != "sys")
        print(f"{conditions} -> n4 {outcome}")
```

```
e3=1 and e4=1 -> n4 survives
e2=1 and e6=1 -> n4 survives
e2=0 and e3=0 -> n4 fails
e4=0 and e6=0 -> n4 fails
e2=0 and e4=0 -> n4 fails
e3=0 and e6=0 -> n4 fails
```

RSR classifies the BN's edge samples; ndtools resolves any unclassified configurations. The 50/50 edge probabilities used during rule search are not the BN's probabilities.

4. Sample the BN

tbnpy samples the system state and all its ancestors using PyTorch. These are Monte Carlo estimates under the demo scenario.

```
from tbnpy import inference

torch.manual_seed(20261022)
state_key = next(k for k, p in probs.items() if p.spec.get("output") == "state")
result = inference.sample(probs, [state_key], n_sample=5000)[state_key]
show_table(pd.DataFrame([
    {"origin": v.name.split("::")[1],
     "P(survives)": f"{(result.Cs[:, j] == 1).double().mean().item():.3f}"}
    for j, v in enumerate(result.childs)
]))
```

origin	P(survives)
n3	0.988
n4	0.853
n5	0.906
n6	0.886
n7	0.998
n8	0.977

Sampling time

CPU timings for the same six system states and their ancestors, using batches of 50,000. Each time is the median of three runs after a warm-up; model building and RSR extraction are excluded.

PyTorch 2.14.1+cpu; 8 CPU threads; float64.

samples	median time (s)	samples/s
100,000	0.234	427,727
200,000	0.393	508,928
300,000	0.640	468,584
400,000	0.772	518,007
500,000	0.972	514,381
600,000	1.154	520,146
700,000	1.352	517,742
800,000	1.558	513,333
900,000	1.873	480,491
1,000,000	2.094	477,669

The individual timings and settings are saved in `tbnpy/demo/sampling_benchmark.json`.